

Cahier Des Charges

Nom du projet :
Collecteurs de données animalières

Version du document :

n°2

Date du document :

4 / 1 / 2026

Client :

Xavier Leprevost - élu de la mairie de Saint-Mars-du-Désert

Auteurs :

Ewen Le Callet

Xueying Xu

Davy Clark

Jules Noumba

Nolan Buchet

Joris Menard

Table des matières

I	Rapport des tests du récepteur JSON	3
1	But	3
2	Résumé des tests	3
3	Tests	3
3.1	fichier json	3
3.2	Recepteur JSON	4
3.3	Émetteur de test	4
3.4	Comparaison des données	4
4	Validation	4
5	Remarques	4
II	Reconstruction des images	5
1	But	5
2	Résumé des tests	5
3	Tests	5
3.1	Liste des formats d'images	5
3.2	Fonction de transformation	5
3.3	Comparaison et validation	6
4	Remarques	6
III	Base de données SQL	7
1	But	7
2	Résumé des tests	7
3	Tests	7
3.1	Tables	7
3.2	Point d'entrée	8
3.3	Insertion de données (exemple)	8
3.4	Affichage web simple	8
4	Remarques	8

I Rapport des tests du récepteur JSON

1 But

Vérifier la validité d'un fichier JSON et l'intégrité des données reçues.

2 Résumé des tests

- Étape 1 : créer un fichier JSON exemple
- Étape 2 : créer un script de test qui importe le récepteur JSON et valide le JSON
- Étape 3 : créer un pseudo émetteur qui envoie le JSON au récepteur
- Étape 4 : comparer les données reçues avec les données envoyées

3 Tests

3.1 fichier json

Dans un premier temps, un fichier JSON exemple a été créé avec les données (spécifié lors de réunions) suivantes :

- IMG : buffer de l'image en sortie de la caméra
- TEMPÉRATURE : température en degrés Celsius
- HUMIDITÉ : humidité en pourcentage
- LONGITUDE : longitude
- LATITUDE : latitude
- HORODATAGE : date et heure
- NIVEAU DE BATTERIE : niveau de batterie en pourcentage

or, d'autres champs ont été rajoutés pour les besoins de reconstruction d'image les champs suivant ont été ajoutés :

- TYPE : type de format de l'image (jpeg, png, etc.)
- WIDTH : largeur de l'image
- HEIGHT : hauteur de l'image

ce fichier json est disponible à : `include/code_json_receiver/data.json`

3.2 Recepteur JSON

le récepteur JSON est disponible à : `include/code_json_receiver/receiver.py`
ce script permet de recevoir un objet JSON depuis une page web avec la méthode POST

3.3 Émetteur de test

le récepteur JSON est disponible à : `include/code_json_receiver/emitter.py`
ce script permet d'envoyer l'objet JSON (défini dans Étape 1) à une page web avec la méthode POST

3.4 Comparaison des données

pour vérifier l'intégrité des données reçues, des JSON simples ont été créés.
JSON envoyé :

```
{  
  "TEMP": 25,  
  "HUMIDITY": 60  
}
```

JSON reçu :

```
{  
  "TEMP": 25,  
  "HUMIDITY": 60  
}
```

4 Validation

les données reçues sont identiques aux données envoyées, le test est donc concluant.

5 Remarques

— Le récepteur JSON a correctement validé les fichiers JSON envoyés.

II Reconstruction des images

1 But

L'objectif de cette étape est de reconstruire les images à partir des données brutes reçues du capteur photo.

2 Résumé des tests

- étape 1 : lister les formats d'images produits par le capteur photo
- étape 2 : mettre en oeuvre la fonction transformant les données brutes en image
- étape 3 : comparer les images reconstruites avec des images de référence

3 Tests

3.1 Liste des formats d'images

Le capteur photo utilisé dans le projet peut produire des images dans les formats suivants :

- PIXFORMAT_RGB565 : 2BPP/RGB565
- PIXFORMAT_YUV422 : 2BPP/YUV422
- PIXFORMAT_YUV420 : 1.5BPP/YUV420
- PIXFORMAT_GRAYSCALE : 1BPP/GRAYSCALE
- PIXFORMAT_JPEG : JPEG/COMPRESSED
- PIXFORMAT_RGB888 : 3BPP/RGB888
- PIXFORMAT_RAW : RAW (tres gros fichier)
- PIXFORMAT_RGB444 : 3BP2P/RGB444
- PIXFORMAT_RGB555 : 3BP2P/RGB555

pour les besoins de ce projet, seuls les formats JPEG, RGB565, GRAYSCALE et RGB888 seront retenus.

3.2 Fonction de transformation

La fonction de transformation sert à transformer les octets de données brutes en une image exploitable. la fonction de transformation est dépendant du format des pixels.

3.3 Comparaison et validation

Validation pour le format JPEG :

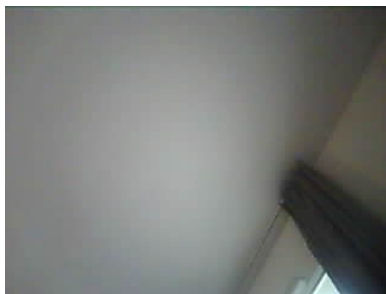


FIGURE II.1 – Image de référence

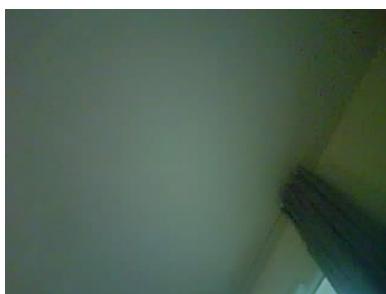


FIGURE II.2 – Image reconstruite

L'image reconstruite est conforme à l'image de référence. Cependant, il y a une légère différence de luminosité entre les deux images.

4 Remarques

- valider le fonctionnement pour les formats d'image RGB565, GRAYSCALE et RGB888
- il faudrait rajouter des corrections de luminosité et de contraste pour améliorer la qualité des images reconstruites

III Base de données SQL

1 But

la base de données SQL est le coeur du work package du stockage distant. Elle doit permettre de stocker les images et leurs métadonnées associées.

2 Résumé des tests

- étape 1 : création des tables
- étape 2 : création d'entrées dans les tables
- étape 3 : insertion d'images (exemple)
- étape 4 : affichage des données sur une page web simple

3 Tests

3.1 Tables

le stockage distant présente 2 tables :

- table "MAIN" : stocke les images reçues et leurs métadonnées
- table "STATS" : statistiques d'apparition d'un certain type d'animal (pas encore implémenté)

3.2 Point d'entrée

3.3 Insertion de données (exemple)

3.4 Affichage web simple

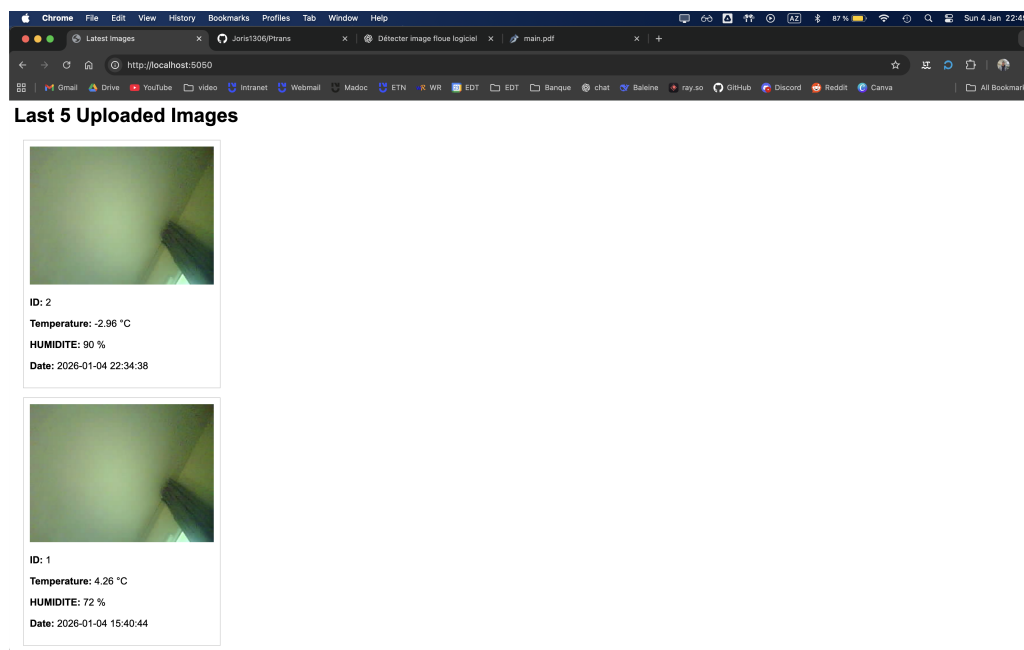


FIGURE III.1 – Affichage web simple des images stockées dans la base de données SQL

4 Remarques

- le WP IHM est lié directement au chapitre 3.4
- créer un nouveau point de sortie pour le WP AI CLASSIFICATION